NRLC-07: Logs, Tags & Drop Rules

> **Series:** NRLC — New Relic to Dynatrace Migration Deep Dives | **Notebook:** 7 of 9 | **Created:** April 2026 | **Last Updated:** 04/15/2026

Overview

The cost-optimization layer of the migration. NR's flat retention and ingest-time drop rules become DT's tiered Grail buckets, OpenPipeline filters, and entity tags. This deep dive covers log forwarding patterns, drop rule conversion, log parsing rule (Grok→DPL) translation, and tag taxonomy migration.

Phase 17 + 18 + 23 + 24 data-side coverage (post-2026-04-15): the engine now auto-converts a broad surface of data-layer configuration beyond drop/parse/tag:

Capability	Transformer	Phase
PII / PAN log obfuscation (7 presets + regex)	`log_obfuscation_transformer`	17
Custom event ingest (bizevent CloudEvent payloads)	`custom_event_ingest_transformer`	17
NR Log Live Archive → Grail bucket + egress (S3/GCS/Azure Blob)	`log_archive_transformer`	24
Metric normalization (rename / aggregate / drop processors)	`metric_normalization_transformer`	24
Database monitoring (10 engines: MySQL / Postgres / MSSQL / Oracle / MongoDB / Redis / Cassandra / MariaDB / DB2 / HANA)	`database_monitoring_transformer`	24

Capability	Transformer	Phase
On-host integrations (12 techs: NGINX / HAProxy / Kafka / RabbitMQ / Elasticsearch / Memcached / Couchbase / Consul / Apache / etcd / Varnish / Zookeeper)	`on_host_integration_transformer`	24
Security Signals / IAST	`security_signals_transformer`	24
Custom entities (NR entity platform → DT custom-device API)	`custom_entity_transformer`	24
Cloud integrations (AWS 16 / Azure 8 / GCP 8)	`cloud_integration_transformer`	18
Kubernetes → DynaKube	`kubernetes_transformer`	18
Prometheus ingestion	`prometheus_transformer`	18
OTel metrics / OTel collector (traces + metrics + logs + 5 processors)	`otel_metrics_transformer` (23) + `otel_collector_transformer` (24)	23/24
StatsD on ActiveGate	`statsd_transformer`	23
CloudWatch Metric Streams (Firehose)	`cloudwatch_metric_streams_transformer`	23
AI Monitoring (model registry + inference NRQL→DQL)	`ai_monitoring_transformer`	18
NPM (SNMP + NetFlow, secrets redacted)	`npm_transformer`	18
Vulnerabilities + per-CVE muting	`vulnerability_transformer`	18

See COVERAGE-MATRIX.md §4, §6, §15, §16 for the complete row-by-row mapping.

Table of Contents

Prerequisites

Requirement	Details
Audience	Platform engineers, FinOps stakeholders
Standalone	This notebook is self-contained for logs/tags/drops migration. No required prerequisite reading.
Optional depth	NRLC-08 (validation), NRLC-09 (toolchain)
Recommended companion	USFOODS-G.02 Grail Buckets (the bucket strategy this maps onto)
Tooling	`Dynatrace-NewRelic`'s `LogParsingTransformer`, `DropRuleTransformer`, `TagTransformer`

Embedded Translation Context — Log Filter & Parsing Patterns

Drop rules and parsing patterns translate predictably between NRQL/Grok and DPL.

Drop rule (NRQL filter → DPL filterOut)

```
-- NRQL
DELETE FROM Log WHERE message LIKE '%health%' AND service = 'frontend'
```

```
-- DPL (OpenPipeline filterOut)
filterOut: contains(content, "health") AND service.name == "frontend"
```

Parsing rule (Grok → DPL)

```
-- Grok
%{IPORHOST:client_ip} - %{NUMBER:response_ms} %{TIMESTAMP_ISO8601:ts}
```

```
-- DPL
IPADDR:client_ip ' - ' INT:response_ms ' ' TIMESTAMP('yyyy-MM-
dd''T''HH:mm:ss.SSS'):ts
```

Tag rule (NR entity tag → OpenPipeline enrichment)

```
# OpenPipeline enrichment that adds environment attribute to ingested logs
matcher: contains(k8s.namespace.name, "prod")
field: environment
value: prod
```

Translation Confidence

Pattern	Confidence	Notes
Simple `WHERE` drop rule	HIGH	Direct DPL filterOut
Drop rule with OR/AND/NOT	HIGH	Boolean operators map directly
Drop rule with regex	MEDIUM	Verify DPL regex equivalent
Standard Grok primitives	HIGH	%{IPORHOST}, %{NUMBER}, %{TIMESTAMP_ISO8601} all supported

Pattern	Confidence	Notes
Custom Grok library	MEDIUM	May need DPL pattern reformulation
Tag rule on entity attribute	HIGH	Becomes OpenPipeline enrichment

1. Log Forwarding Patterns

NR's primary log ingest paths:

Source	NR Endpoint	DT Equivalent
Filebeat / Fluent Bit	NR Log API	OneAgent log ingest or Fluent Bit → OpenPipeline
Lambda log forwarder	NR Lambda extension	Dynatrace Lambda extension or CloudWatch → Firehose → OpenPipeline
K8s log integration	NR Kubernetes integration	DynaKube + log monitoring
Syslog	NR Syslog forwarder	OpenPipeline syslog ingest endpoint
HTTP/JSON	NR Log API	DT Generic Log Ingest API

Migration: reconfigure each forwarder to point at DT. Run dual-shipping (both NR and DT receive) for the dual-run period; cut NR after volume confirmation.

2. Drop Rules → OpenPipeline Filters

NR drop rules filter at ingest. DT's equivalent is OpenPipeline's `filterOut` processor.

NR drop rule (NRQL filter):

```
DELETE FROM Log WHERE message LIKE '%health%' AND service = 'frontend'
```

Equivalent OpenPipeline rule (DPL):

```
filterOut: contains(content, "health") AND service.name == "frontend"
```

The `DropRuleTransformer` translates each NR drop rule to an OpenPipeline filter rule, preserving:
- the filter expression (NRQL → DPL)
- the source identification (NR data type → DT data table)
- the rule's enabled/disabled state

Translation is HIGH confidence for simple `WHERE` clauses; MEDIUM for clauses with regex or nested boolean logic.

3. Log Parsing — Grok → DPL

NR uses Grok patterns for log parsing; DT uses DPL (Dynatrace Pattern Language).

Both are pattern languages but with different syntax. The `LogParsingTransformer` handles common Grok primitives:

Grok	DPL
`%{IPORHOST:client_ip}`	`IPADDR:client_ip`
`%{NUMBER:duration_ms}`	`INT:duration_ms` or `DOUBLE:duration_ms`
`%{TIMESTAMP_ISO8601:timestamp}`	`TIMESTAMP('yyyy-MM-dd''T''HH:mm:ss.SSS'):timestamp`
`%{WORD:method}`	`LD:method` (with constraint)
`%{DATA:rest}`	`LD:rest`
`%{GREEDYDATA:everything}`	`LD:everything`

Conversion confidence:

- HIGH for patterns built from common Grok aliases
- MEDIUM for patterns with custom Grok library imports
- LOW for patterns relying on Grok's regex-anywhere capability (DPL is more positional)

DPL parses are part of OpenPipeline; once converted, they apply at ingest before data lands in the bucket.

4. Tag Taxonomy Migration

NR has account-wide tags applied to entities. DT has both **entity tags** (applied to discovered entities) and **Grail dimensions** (applied to ingested data).

NR Tag Type	Gen3 DT Equivalent
Entity tag (e.g., `env:prod` on a host)	OpenPipeline enrichment that adds the same attribute to incoming data (e.g., `environment = "prod"`)
Workload tag	OpenPipeline enrichment emitting `workload.name = "<value>"` + optional bucket routing
Account-level tag	OpenPipeline enrichment scoped by ingest source / bucket

(Note: Gen2 entity tags and auto-tagging rules still exist, but the canonical Gen3 pattern enriches data at ingest so the attribute is queryable in DQL and usable in IAM bucket conditions.)

Recommended approach: convert NR tags to **OpenPipeline enrichment rules**. New ingested data automatically receives the attribute as it lands in Grail, so DQL queries and IAM bucket-scoping work without depending on Gen2 entity tagging.

```
# OpenPipeline enrichment example
matcher: contains(k8s.namespace.name, "prod")
field: environment
value: prod
```

The `TagTransformer` emits one OpenPipeline enrichment rule per unique NR tag pattern. (Gen2 auto-tagging is also supported for hybrid tenants but is not the recommended Gen3 approach.)

5. Cost Optimization Patterns

Migration is a chance to reset the cost profile. Common patterns:

Optimization	Approach
Drop noisy health-check logs	OpenPipeline drop rule (catches 15–25% of log volume)

Optimization	Approach
Drop DEBUG in production	OpenPipeline drop rule scoped to prod namespaces
Tier infra logs to 14 days	Route to a 14-day bucket (vs. 35-day default)
Tier compliance logs to 365 days	Route to a usage-based pricing bucket
Extract metrics from logs	OpenPipeline metric extraction (avoids querying raw logs)
Sample low-value high-volume logs	OpenPipeline sampling processor

Combined, these typically yield **30–60% lower DPS spend** than a default-bucket-only deployment. See **USFOODS-G.02 Grail Buckets** for the full design pattern.

6. Validation Queries

After log migration, run these DQL queries to confirm:

Volume parity (DT vs. NR)

```
fetch logs, from:-1d
| summarize records = count(), by:{dt.system.bucket}
| sort records desc
```

Compare against NR's `SELECT count(*) FROM Log SINCE 1 day ago` for the same window. Volumes should match within ±5%.

Drop rule effectiveness

```
fetch logs, from:-1h
| filter contains(content, "health")
| summarize count()
```

If the drop rule is working, this should return 0 (or near-zero).

Parsing extraction

```
fetch logs, from:-1h
| filter isNotNull(client_ip)
| summarize count(), by:{client_ip}
| sort count() desc
| limit 10
```

If parsing is working, parsed fields should be non-null.

Tag application (Gen3 — DQL on enriched data)

```
fetch logs, from:-1h
| filter environment == "prod"
| summarize count(), by:{environment}
```

Should return enriched data tagged with the expected environment value. (For Gen2 entity-tag verification, `fetch dt.entity.host | filter tag("env") == "prod"` still works on hybrid tenants but does not query Grail data directly.)

Summary

The logs/tags/drops layer is where migration pays for itself in cost terms. Drop rules port directly; Grok→DPL parsing is mostly mechanical; tags become OpenPipeline enrichment rules that attach attributes to ingested data (Gen3 pattern). Combined with the bucket strategy in USFOODS-G.02, expect 30–60% cost reduction.

Continue to **NRLC-08 Validation, Diff & Rollback** for the verification layer.

Tooling for Logs-Only Migration

```
# 1. Inventory NR log forwarding configs, drop rules, parsing rules, tags
python3 migrate.py migrate --export-only --components logs,drops,parsing,tags
--output ./logs-export

# 2. Translate (Grok → DPL; tag rules → OpenPipeline enrichments; drops →
OpenPipeline filters)
python3 migrate.py migrate --transform-only --components
logs,drops,parsing,tags --report

# 3. Diff
python3 migrate.py migrate --diff --components logs,drops,parsing,tags

# 4. Apply OpenPipeline configurations to DT tenant
python3 migrate.py migrate --import-only --components logs,drops,parsing,tags

# 5. Reconfigure log forwarders (Filebeat / Fluent Bit / Lambda extension) to
point at DT
# 6. Dual-ship for 1–2 weeks; compare volume and confirm drops working
```

This notebook was AI-generated from community-submitted and publicly available sources, including the open-source Dynatrace-NewRelic, nrql-engine (planned future home: the `dynatrace-dma` Dynatrace Migration Assistant organization), and nrql-translator projects. This notebook series is not officially supported by Dynatrace or New Relic. Always verify information against the official Dynatrace documentation and New Relic documentation.